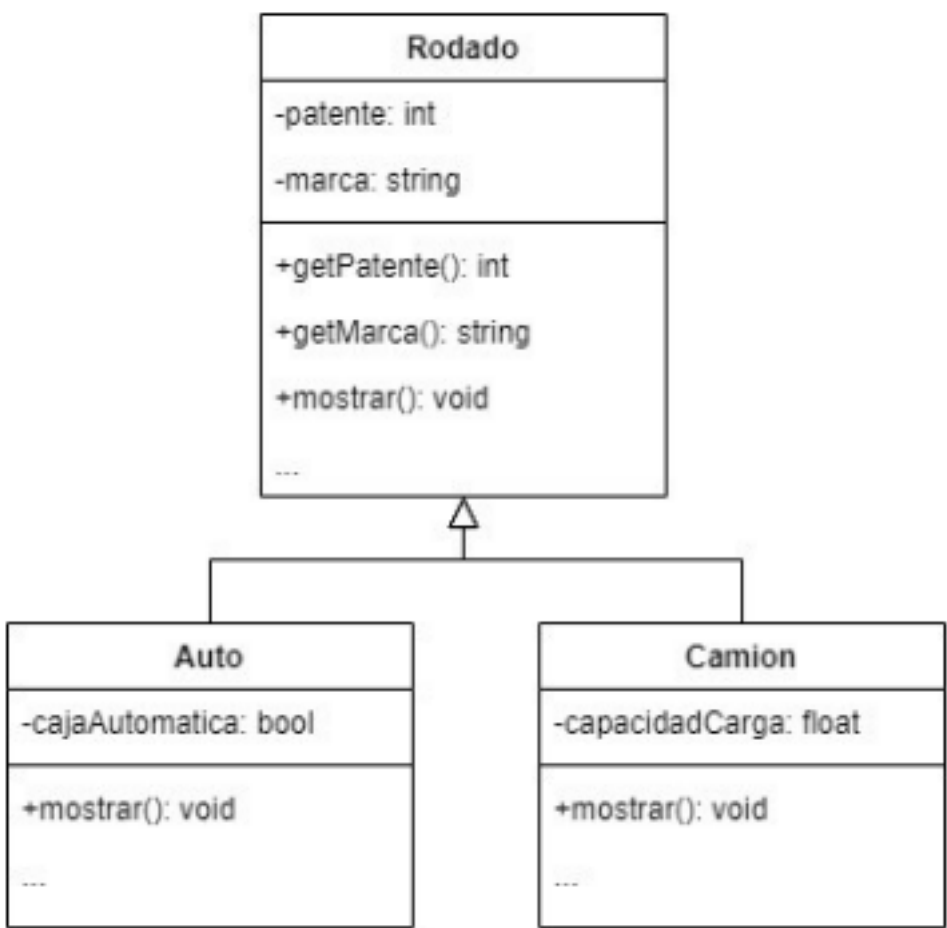


Introducción a la POO

Pilares de la Programación Orientada a Objetos:

- Encapsulamiento
- Herencia
- Polimorfismo

Herencia



- class ClaseHija extends ClasePadre {}

clase de la cual
hereda

dase que hereda

Superclase

Subclase

Padre

Hijo

Antecesor

Sucesor

Ancestro

Descendiente

clase superior

Especialización

clase base

clase derivada

Permite crear nuevas clases a partir de otras.

ventajas: reutilización de código, jerarquía lógica.

Visibilidad del miembro en la clase-base	Modificador de acceso utilizado en la declaración de la clase derivada		
	public	protected	private
public	public	protected	private (accesible)
protected	protected	protected	private (accesible)
private	private (no accesible directamente)	private (no accesible directamente)	private (no accesible directamente)

Ejemplo de herencia

```
class Animal
{
    void comer() {
        System.out.println("Este animal come");}
}
class Perro extends Animal
{
    void ladrar() {
        System.out.println("El perro ladra"); }
}
```

Constructores y super

Se puede llamar al constructor padre usando `super()`.

```
class Gato extends Animal
{
    Gato()
    {
        super();
        System.out.println("Gato creado");
    }
}
```

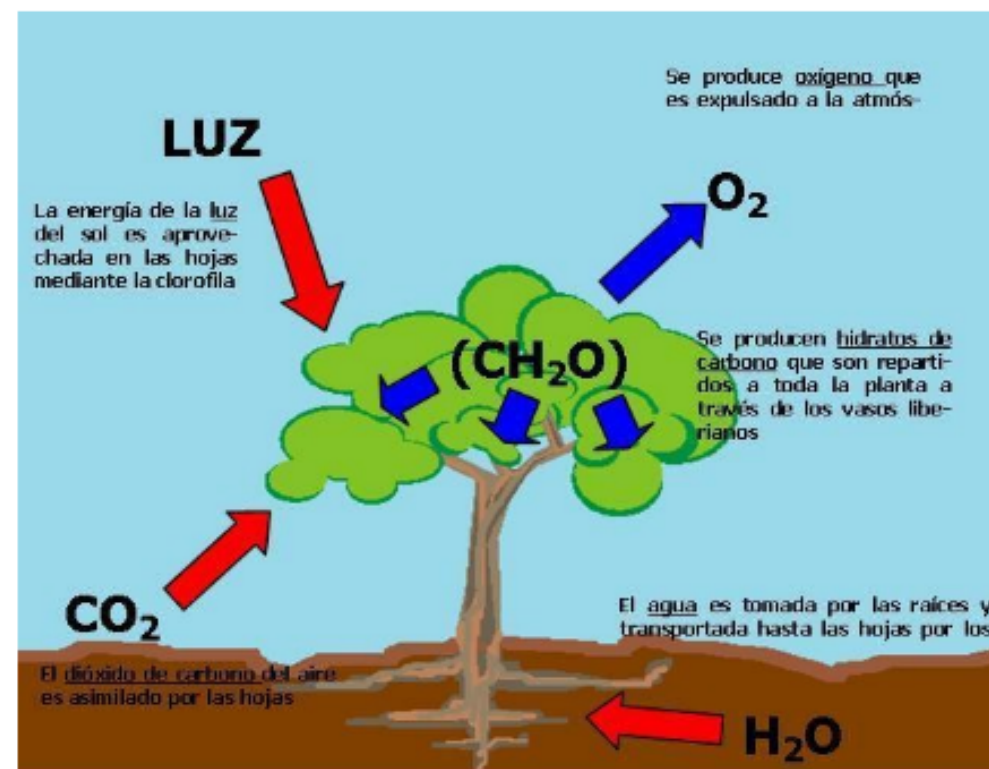
Sobrescritura de métodos

@Override Permite redefinir un método heredado.

@Override permite redefinir un método heredado.

```
class Gato extends Animal
{
    @Override void hacerSonido()
    {
        System.out.println("Miau");
    }
}
```


Polimorfismo



Capacidad de un objeto de tomar múltiples formas.

Tipos:

- En tiempo de compilación (sobrecarga).
"Polimorfismo estático", ocurre cuando varios métodos tienen el mismo nombre, pero diferentes parámetros (tipo, cantidad o ambos).
- En tiempo de ejecución (sobrescritura).
"Polimorfismo dinámico", ocurre cuando una subclase redefine un método heredado de su clase padre.

Ejemplo de polimorfismo

```
Animal a1 = new Perro();
Animal a2 = new Gato();
a1.hacerSonido(); // Guau
a2.hacerSonido(); // Miau
```

Clases Abstractas

- No se pueden instanciar directamente
- Métodos abstractos obligan a la subclase a implementarlos.

```
// Clase abstracta
abstract class Figura {
    String color;

    // Constructor
    Figura(String color) {
        this.color = color;
    }

    // Método abstracto (sin implementación)
    abstract double calcularArea();

    // Método normal
    void mostrarColor() {
        System.out.println("Color: " + color);
    }
}
```

```
class Circulo extends Figura {
    double radio;

    Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override
    double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```


Interfaces

Es un contrato que define qué debe hacer una clase, pero no cómo.

Antes de Java 8 no podía tener implementación.

Desde Java 8 en adelante, puede tener:

- Métodos abstractos
- Métodos default (con implementación)
- Métodos static
- Constantes (public static final)

Clase abstracta vs Interfaz

<u>Característica</u>	<u>Clase abstracta</u>	<u>Interfaz</u>
<u>Puede tener atributos</u>	✓ <u>Sí</u>	⚠ Solo <u>constantes</u> (static final)
<u>Métodos con implementación</u>	✓ <u>Sí</u>	✓ <u>Desde Java 8</u> (default)
<u>Constructor</u>	✓ <u>Sí</u>	✗ No
<u>Herencia múltiple</u>	✗ No	✓ <u>Sí</u> (una clase puede <u>implementar varias</u>)
<u>Uso típico</u>	"Es <u>un tipo de...</u> "	" <u>Puede hacer...</u> "

Ejercicio 1

- Desarrollar:
- clase base: Vehiculo
 - clases derivadas: Auto, Moto
 - Método mover() sobrescrito en cada clase
 - Llamar a mover() desde una lista de Vehiculo

Ejercicio 2

Simular un sistema de transporte donde distintos vehículos calculan su costo de viaje en base a su tipo.

- 1) Crear una clase abstracta Vehiculo con un método abstracto `calcularCostoViaje (double distancia)`
- 2) Crear las clases Auto, Moto y Colectivo que extiendan de Vehiculo. Cada una debe implementar su propio cálculo del costo (por ejemplo: auto = 10 \$ / km, moto = 5 \$ / km, colectivo = 3 \$ / km).
- 3) En una clase Terminal, crear una lista de vehículos y mostrar el costo de un viaje de 50 km para cada uno.

Ejercicio 3

Desarrollar un sistema de evaluación para distintos tipos de estudiantes. Dependiendo de si el estudiante es de grado o posgrado, el cálculo de su nota final puede variar.

- 1) Crear una clase Estudiante con atributos comunes como nombre y un método `calcularNotaFinal()`.
- 2) crear dos subclases:
 - Estudiante Grado: la nota final es el promedio de 3 parciales
 - Estudiante Posgrado: la nota final es el 70% del trabajo final y el 30% de un examen oral.
- 3) En una clase evaluador, instanciar varios estudiantes de grado y posgrado, calcular y mostrar sus notas finales.